Logic Restructuring with Preserved Logic Blocks

Siang-Yun Lee
Cadence Design Systems,
Munich, Germany

Heinz Riener
Cadence Design Systems,
Munich, Germany

Sascha Richter
Cadence Design Systems,
Munich, Germany

Ankush Sood
Cadence Design Systems,
San Jose, CA, USA

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11132654

***Abstract*—During technology mapping, complex cells such as adders and multiplexers are often available in the standard cell library, which helps improve the final PPA results. However, technology-independent optimization tends to over-optimize towards the cost metrics measurable with technology-independent representations (e.g., AIG size) by decomposing blocks of logic that could have been mapped into complex cells. Besides, it is also of practical interest to model and preserve some special logic components, such as the enable logic of flops, during optimization. This paper studies "boxing" logic blocks before technology-independent optimization and preserving them during optimization. A wire-based resubstitution is proposed to optimize boxed networks. Experiments with academic and industrial benchmarks show that transparent-boxing, i.e., preserving boxes while utilizing the information on their logic, achieves better results than black-boxing, i.e., completely ignoring the logic in boxes. Specifically, in the technology-independent evaluation, transparent-boxing reduces the mapped network size by 4% more than black-boxing; in the full-flow evaluation, transparent-boxing achieves 1.8% more improvement on timing and similar improvements in area, power, and wire length compared to black-boxing.**

***Index Terms*—Logic synthesis, combinational circuits, Boolean optimization**

I. Introduction

Due to scalability concerns, modern logic synthesis flows are typically separated into technology-independent optimization, which focuses on removing logical redundancies using a homogeneous network data structure like the *AND-Inverter Graphs* (AIGs) [1], and technology mapping [2], which converts the optimized *subject graphs* (e.g. AIGs) into mapped netlists using a technology-specific standard cell library. In technology mapping, the subject graph is *covered* by blocks of logic available in the library by structural and logical analysis. When multiple possibilities exist, choices are made to optimize towards cost metrics measurable using information provided in the standard cell library, such as cell area, power, and delay. In contrast, in technology-independent optimization, choices are made to minimize rather simple and approximated cost metrics, such as the number of AIG nodes and the AIG depth.

While the contents of different standard cell libraries differ, some special, commonly used cells are often available, such as half adders and full adders. A *half adder* (HA) is a two-input, two-output cell computing the following functions:

$$\mathrm{HA}_c(x, y) = x \wedge y$$
$$\mathrm{HA}_s(x, y) = x \oplus y.$$

A *full adder* (FA) is a three-input, two-output cell computing the following functions:

$$\mathrm{FA}_c(x, y, z) = (x \wedge y) \vee (y \wedge z) \vee (x \wedge z)$$
$$\mathrm{FA}_s(x, y, z) = x \oplus y \oplus z.$$

Empirical observation has been found that correctly utilizing adder cells, instead of composing the same functions using elementary logic gates, may improve the *power, performance, and area* (PPA) results of the final chip [3]. As another example, modeling the sequential enable and set/reset logic of flip-flops as complex flops enhances the synthesis quality, e.g., demonstrated in [4] for an FPGA synthesis flow. A complex flop composed of a simple flop (with inputs D, *clk*, and output Q) and one or two MUXes connected to the D input of the simple flop. To model the sequential enable logic, the MUX computes

$$D = (\mathit{en} \wedge d) \vee (\neg \mathit{en} \wedge Q),$$

where *en* is the sequential enable and d is the data. To model the sequential set/reset logic, the MUX computes

$$D = (\mathit{rst} \wedge \mathit{init}) \vee (\neg \mathit{rst} \wedge d),$$

where *rst* is the sequential reset and *init* is the initial value. If both the enable and the set/reset logic are to be modeled, the two MUXes are concatenated. To not destroy the complex flop, one must guarantee that the MUXes are preserved during logic optimization. Besides these, some other cells or logical components are also of practical interest to be modeled and preserved in logic synthesis, such as clock gating logic [5], pre-mapped user hierarchical instances, and components with regular structures (e.g. XORs and MUXes) [6].

It is thus desired to preserve some special logic blocks exemplified above during optimization. However, without intentionally protecting these special logic blocks, classic technology-independent optimization algorithms have a high chance of destroying them structurally, making it difficult to restore or re-discover them later in technology mapping. This problem is tackled in some open-source and industrial tools by *black-boxing* special logic blocks. When a block is black-boxed, AIG nodes representing its function are removed, creating an empty hole in the network whose interfaces are connected to additional virtual PIs and POs to keep the graph acyclic and non-dangling. However, to correctly identify the critical path, one needs to annotate the virtual PIs with additional level information and update this information if the logic changes during optimization. While the black-boxing strategy successfully protects these blocks, we argue that a *transparent-boxing* strategy that does not neglect the logical information within boxes would be more advantageous. In a transparent-boxing strategy, boxed nodes are kept in the network and are utilized to, for example, compute don't care conditions for other nodes. In the context of critical path identification, it is easier to update the level information at the transparent box outputs when their input levels are updated.

In order to take advantage of the transparent boxes, we need to develop new algorithms or adjust existing logic synthesis algorithms. In this context, we present a new optimization algorithm, called wire-based resubstitution that, in contrast to existing node resubstitution techniques, focuses on input pins of transparent boxes. To leverage the additional information provided by the transparent boxes, observability don't cares (ODCs) play an important role in this algorithm. Since nodes in the transparent boxes have to be preserved, optimization is only possible at the input pins, whereas the boxed nodes contribute to the ODC computation. As the node connected to an input pin may have other fanouts outside of the box, the proposed wire-based

resubstitution is necessary to optimize the box where existing node-based methods cannot.

Our experiment on technology-independent evaluation with the EPFL benchmarks shows that transparent boxing achieves the most reduction in the number of free ANDs as well as in the estimated mapped network size, compared to black boxing and no boxing. Experiments on industrial benchmarks using integration with an industrial synthesis tool show improvements of 9.8% in total negative slack (TNS), 0.5% in area, 1.8% in power, and 2.7% in wire length with black-boxing. On top of such, with the addition of the proposed wire-based resubstitution and transparent boxing, 1.8% more improvement in TNS and similar improvements in area, power, and wire length are observed.

In summary, the contributions of this paper are three-fold:

1) Detailed explanations on how boxing techniques are efficiently implemented, especially for transparent-boxing.
2) A novel wire-based resubstitution algorithm for optimizing transparent-boxed networks.
3) Experimental evaluation on both academic and industrial benchmarks showing PPA improvements given by the proposed methods, demonstrating the importance of boxing and preserving special logic blocks.

II. Preliminaries

A. Logic Networks

Logic networks are technology-independent representations used in logic synthesis. A *network* is a *directed acyclic graph* (DAG) where nodes model simple logic gates and edges model wires potentially with integrated inverters (fanin negations). The *primary inputs* (PIs) and the *primary outputs* (POs) are the interfaces between a network and the outside environment. A representative example of logic networks is the *AND-Inverter Graph* (AIG) where each node is a two-input AND gate. In the figures in this paper, wires with inverters are drawn as dashed edges.

In a typical AIG data structure, a list of nodes is stored, where each node stores pointers to its two fanin nodes, together with fanin negation tags. PIs are represented by special nodes, identified by having identical fanins. POs are stored separately as a list of pointers to nodes with negation tags. During AIG construction and throughout optimization, trivial cases are checked and simplified such that the following conditions should never happen: (1) A node has a constant (0 or 1) fanin. (2) A non-PI node's two fanins point to the same node (with or without the same fanin negation tag). Whenever these cases appear, the node should not be created and should be replaced with either a constant or its other fanin. Such replacement and simplification may propagate towards the transitive fanouts.

Moreover, a technique called *structural hashing* is often adopted to avoid redundancies. A hash table is used to quickly look up a node using its fanins (negation tag considered) and to ensure that there is not a second node with exactly the same fanins, i.e., a node is unique to its fanins. During network modification (e.g., node substitution), some nodes' fanins may change, and they should be re-hashed, potentially becoming identical to another existing node and thus merged. Such changes may propagate towards the transitive fanouts.

B. Don't-Cares

Don't cares are flexibilities in a network and are often computed and leveraged in logic optimization to enhance optimization power. They indicate where the local function of a node in the network is allowed to be modified without affecting the global functionality the network computes. Two types of don't-cares are typically considered.

First, for a set of internal nodes, there might be some value combinations that never appear at these nodes. For example, an AND gate g_1 and an OR gate g_2 sharing the same fanins can never have $g_1 = 1$ and $g_2 = 0$ at the same time. This combination is a *satisfiability don't-care* (SDC) for a common transitive fanout of g_1 and g_2.

Second, a value assignment $\vec{x} \in \mathbb{B}^n$ to the PIs is said to be *unobservable* with respect to a node n if none of the POs changes its value when n is replaced by its negation $\overline{n}$. $\vec{x}$ is an *observability don't-care* (ODC) of n because the function of n under $\vec{x}$ does not matter.

Typically, SDCs are computed at the transitive fanins of the node under optimization, whereas ODCs are computed using the transitive fanouts of the node.

C. Simulation-Guided Resubstitution

Boolean resubstitution is a technology-independent logic optimization algorithm that attempts to *resynthesize* the *maximum fanout-free cone* of a *root* node using existing *divisor* nodes having common support as the root. In cut-based resubstitution, divisors are collected within a smaller window supported by a cut of size 8-10 (typically no larger than 16), and the resynthesis is performed with (complete) truth tables exhaustively simulated from the cut. In contrast, in simulation-guided resubstitution [7], partial truth tables computed from the primary inputs using a non-exhaustive set of simulation patterns are used to approximate nodes' global functions. One of the advantages of simulation-guided resubstitution is that global SDCs are considered implicitly with global simulation without needing extra effort to compute them. It is also possible to leverage ODCs in simulation guided resubstitution. When doing so, ODCs are often computed only up to 5 levels of the transitive fanout cone because involving too large fanout cones would result in a high runtime.

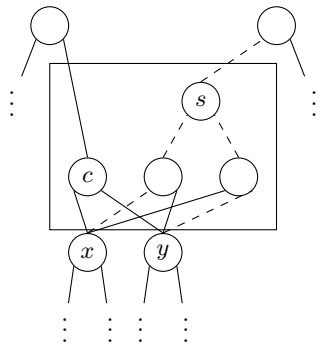
D. Black-box and Transparent-box Testing

In the field of software testing, the term *black-box testing* refers to a testing strategy that examines the functional correctness of software according to its input-output relationship without knowledge of its internal workings. In contrast, *transparent-box testing* (or *white-box testing*, *clear-box testing*, *glass-box testing*) tests the internal structures of software.

In the context of logic synthesis, we use the term *boxing* to mention the extraction of blocks of logic in a technology-independent representation that can be mapped into (complex, potentially multi-output) cells. We borrow the term *black-boxing* to mention the strategy where the boxed logic is taken away with a "hole" of some dangling wires remaining in the network. In contrast, we use the term *transparent-boxing* for the strategy where the logic function and the internal implementation of the boxes are known and can be utilized during optimization.

III. Boxing Technology-Independent Logic Networks

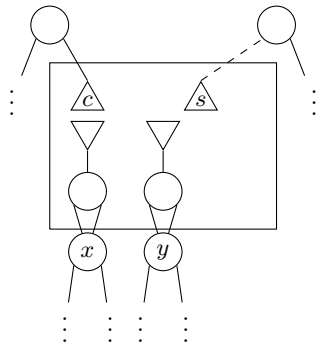
To preserve specific logic blocks in the network during logic optimization, we propose to create boxes to protect them. A *box* in a network is a connected subset of nodes. Two boxes in the same network shall not overlap. A box can be identified by a set of *box inputs*, which are nodes outside the box having a fanout inside the box, and a set of *box outputs*, which are nodes inside the box with at least one fanout outside the box. Optionally, a label referring to the logic function(s) computed by the box outputs in terms of the



Box inputs $= \{x, y\}$

Box outputs $= \{c, \neg s\}$

Fig. 1: A boxed half adder in a network.



Box inputs $= \{x, y\}$

Box outputs $= \{c, \neg s\}$

Fig. 2: Data structure implementation of a black box in a network.

box inputs may be associated with the box. For example, Figure 1 shows a half adder box in a network. In this section, we discuss how black and transparent boxes can be implemented in an AIG-like data structure.

A. Black-Boxing

In a black-boxing strategy, nodes within the box are taken away from the network, and the box's inputs and outputs become dangling. To keep the network a valid DAG, additional virtual PIs are added and used to replace the box outputs, and additional virtual POs are added and connected to the box inputs. Practically, we insert additional buffers (nodes with identical fanins) between the box inputs and the added virtual POs to easily identify these POs as box inputs, in case the box inputs are also connected to a real PO of the network. Figure 2 illustrates such a black-boxing strategy.

Classic logic optimization algorithms can be applied directly on a black-boxed network without incurring algorithmic or correctness errors (e.g., the optimized network becomes cyclic or functionally non-equivalent to the original network). This is because (1) the box outputs are replaced with independent PIs, so no assumptions on their values in relation to other nodes in the network would be made, and (2) the box inputs are connected to additional POs, so their functions would be kept the same under all circumstances.

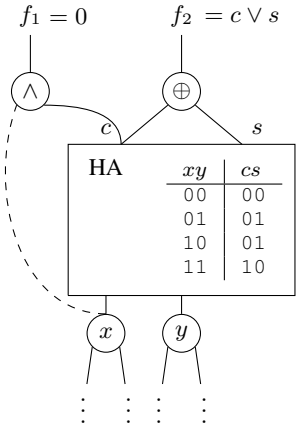


Fig. 3: Example of optimization with boxes.

B. Transparent-Boxing

A transparent-boxing strategy keeps track of, in addition to the interfacing box inputs and outputs, the logic function computed by the box and the internal implementation of the box. There are several ways of storing the box function. For example, storing truth tables, using binary decision diagrams (BDDs), or linking to separated sub-networks. To minimize the impact on the network data structure and to facilitate the adaptation of existing algorithms to boxed networks, we propose to simply keep the boxed nodes in the network but mark them as *don't touch*es. This is an extension to the standard AIG conventions introduced in Section II-A, where some conditions that should not appear in standard AIGs become possible in the transparent boxes.

A don't touch is an additional flag stored for each node in the network. Network modification methods, such as node substitution, are modified to avoid changing nodes marked as don't touch. Special attention should be paid to handling trivial cases, structural hashing, and their propagation towards transitive fanouts. Specifically, (1) constants and repeated fanins can be allowed at box inputs; decisions on whether to simplify the box can be made in a post-optimization process. (2) If a node in a box is (or becomes) structurally identical to a node outside the box, both of them should be kept.

The advantage of this transparent-boxing strategy is that the network data structure remains the same as its non-boxed counterpart, so most non-modifying computations, such as simulation, cut enumeration, and structural analysis, can be done in the same way. An illustration of a white-boxed data structure would look the same as in Figure 1, except that nodes in the box are tagged as don't touches.

IV. Logic Optimization with Boxes

Using a black-boxing strategy, special logic blocks are easily preserved during technology-independent logic optimization. This strategy has the advantage of being straightforward and guarantees to work with any optimization algorithm without adaptation. However, the logical relations between the box inputs and outputs are completely neglected during optimization, which may lead to oversight of potential optimization opportunities. Moreover, by connecting box inputs and outputs to virtual POs and PIs, the overall structure (or the "shape") of the network is changed. For example, a box may be originally "deep" in the network, i.e., having a large logic depth from PIs, but the depth of the box outputs will become very small after black-boxing because they are connected to virtual PIs which are

usually assigned to level 0. While this distortion can be mitigated by assigning non-zero input levels to these PIs according to their real arrival times, this information may still become inaccurate during optimization. When a black box input's level changes due to an optimization made, this change cannot be propagated immediately and correctly to the black box's outputs. As another example, due to the "holes" and disconnections caused by black-boxing, some cuts may disappear. These structural changes may have an impact on the quality of the results of some optimization algorithms.

To avoid the disadvantages of black boxing, we propose to keep the structural and logical information with transparent boxing. Using the proposed transparent-boxing strategy, most optimization algorithms should perform the same as without boxing when optimizing non-boxed nodes. However, some optimization algorithms may benefit more from the information provided by the transparent boxes because they leverage don't-care computations in larger windows beyond the MFFC of the node being optimized. One prominent example of such algorithms is simulation-guided resubstitution [7], which relies on whole-circuit simulation to approximate nodes' global functions, where global satisfiability don't cares are implicitly considered. Another example is the new rewriting framework capable of using don't cares [8].

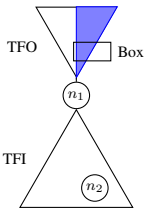
Figure 3 shows two examples of optimizations that are only discoverable with transparent boxing. First, the output function f_1 depends on a box output and a box input, which are related. With a black box in between, optimization algorithms cannot recognize that f_1 always evaluates to constant 0. In contrast, with a transparent box, an algorithm capable of computing don't cares would realize the satisfiability don't care condition $x = 0, c = 1$ and use this fact to substitute the AND gate with constant 0. Second, the output function f_2 depends on two outputs of the same box, which are also related. Again, with a black box model, the two box outputs are unrelated virtual PIs and cannot be optimized. However, with a transparent box, the don't care condition $c = 1, s = 1$ can be computed and used to optimize the XOR gate to an OR gate, which may have a lower cost.

A. Wire-Based Resubstitution

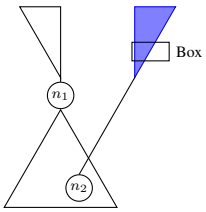
In addition to the existing algorithms suitable for optimizing transparent-boxed networks, in this section, we further devise an algorithm tailored for this purpose. The proposed algorithm aims to complement the other optimizations when being integrated together in a flow by targeting especially on rewiring the input pins of boxes. To better utilize the information provided by the transparent boxes, and also because optimizations happen at box inputs, ODCs are computed and play an important role in this algorithm.

The basic idea is similar to simulation-guided resubstitution, except that instead of substituting a node with another, we try to substitute an input pin of a box with another node in its transitive fanin cone. In other words, instead of requiring functional equivalence of two nodes (subject to don't cares), we seek unchanged functionality at the outputs in the partial transitive fanout cone of a node that passes through the considered box.

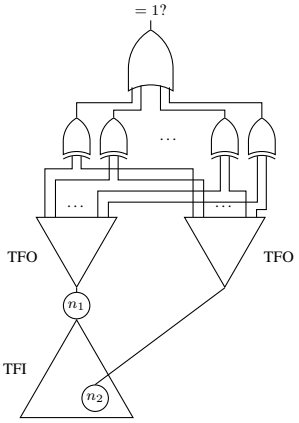
The difference between classic resubstitution and wire-based resubstitution is illustrated in Figure 4. Figure 4a shows the original network with a transparent box at the fanout of node n_1. Part of the transitive fanout cone (TFO) of n_1 passes through the box (blue), whereas the other part does not. A node n_2 in the transitive fanin cone (TFI) is being considered as a divisor to substitute n_1 with. In classic resubstitution considering ODCs, the algorithm checks if substituting n_1 with n_2 would result in any difference in the TFO with a miter of the original TFO and a duplicated TFO connected



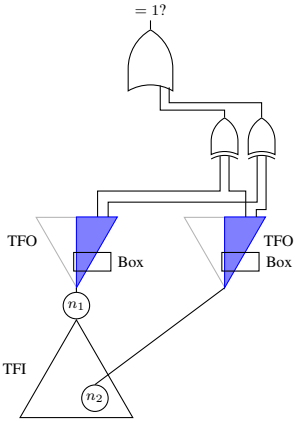
(a) Original network with a transparent box.



(b) Optimized by wire-based resubstituion.



(c) Classic resubstitution considering ODCs.



(d) Wire-based resubstitution.

Fig. 4: Illustrations of ODC-based optimization.

to n_2 (Figure 4c). In contrast, wire-based resubstitution focuses on the partial TFO passing through the box (Figure 4d). Since a smaller part of the network is being affected, wire-based resubstitution has a higher chance to find valid optimizations.

As there may be other fanouts that still require n_1 after substitution, reduction in AIG nodes is not guaranteed. However, we only consider substituting with a node in the TFI, so the move cannot make the AIG size or depth worse. The metrics either stay the same or get better. In summary, the proposed wire-based resubstitution is a relatively light-weight optimization that does not consume much runtime. Nevertheless, it finds some key optimizations and complements other higher-effort optimizations that cannot take full advantage of the transparent-box information.

V. Experimental Results

A. Technology-Independent Evaluation

In this experiment, we implement the boxing techniques discussed in this paper in the C++ logic synthesis library `mockturtle`[1] [9], [10] and use the EPFL benchmark suite [11] for the technology-independent evaluation of the two boxing techniques discussed in this paper. Since these benchmarks are given in form of AIGs and do not contain high-level (RTL) information that may give rise to logical components to be boxed, we perform adder identification to find half adders and full adders. The benchmarks are pre-processed with the balancing command `&b` in ABC [12] as a heuristic to increase the number of recognizable adders. Before optimization, half adders and full adders are identified and boxed by enumerating and analyzing the functions of 3-cuts [13].

[1] Available: https://github.com/lsils/mockturtle

TABLE I: Evaluation of different boxing techniques on EPFL benchmarks.

	Before optimization		Without boxing		Black-boxing			Transparent-boxing		
Bench.	A	A+B	A+B	Time	A	A+B	Time	A	A+B	Time
adder	0	128	128	0.0	0	128	0.0	0	128	0.0
bar	3214	3214	3155	0.0	3155	3155	0.0	3155	3155	0.0
div	13784	17695	17615	2.7	13580	17436	0.4	13575	17518	1.8
hyp	113550	133666	114507	98.1	109682	122860	2.9	108642	113954	63.6
log2	20527	21981	21483	16.8	20451	21915	0.4	19822	21300	2.5
max	2865	2865	2862	0.0	2863	2863	0.0	2863	2863	0.0
multiplier	15384	16895	16826	2.1	15381	16890	0.5	15315	16824	0.2
sin	3554	3820	3743	1.1	3520	3787	0.1	3447	3728	1.5
sqrt	14734	16553	16467	5.2	14615	16438	0.1	14595	14602	2.7
square	7949	9327	8175	0.7	7238	8388	0.5	7067	8169	0.6
arbiter	11839	11839	11839	0.3	11603	11603	0.4	11603	11603	0.4
cavlc	672	679	616	0.2	633	643	0.3	598	614	0.2
ctrl	165	166	93	0.0	112	114	0.0	100	103	0.0
dec	304	304	304	0.0	304	304	0.0	304	304	0.0
i2c	1187	1191	1084	0.0	1105	1110	0.1	1089	1094	0.1
int2float	226	227	212	0.1	209	210	0.2	212	214	0.2
mem_ctrl	46543	46584	38083	2.7	40113	40150	4.3	38298	38353	4.2
priority	843	843	518	0.0	560	560	0.0	560	560	0.0
router	133	171	133	0.0	133	171	0.0	119	164	0.0
voter	1231	2558	2037	1.0	1231	2544	0.0	528	1493	0.6
Total	258704	290706	259880 (-10.6%)	131.0	246488 (-4.7%)	271269 (-6.7%)	10.2	241892 (-6.5%)	256743 (-11.7%)	78.7

TABLE II: Evaluation of different boxing techniques on industrial benchmarks.

	Without boxing	Black-boxing		Transparent-boxing	
		Avg.	W/L	Avg.	W/L
TNS	±0% (baseline)	-9.78%	7/6	-11.55%	8/5
WNS	±0% (baseline)	+3.57%	7/4	+1.19%	8/5
Area	±0% (baseline)	-0.48%	10/2	-0.52%	10/3
Power	±0% (baseline)	-1.75%	10/3	-1.27%	10/3
WL	±0% (baseline)	-2.73%	13/0	-2.36%	12/1

In Table I, statistics are reported for benchmarks before optimization, optimized normally without boxing, optimized with black-boxing, and optimized with transparent-boxing. In all cases, the optimization algorithm is (classic) simulation-guided resubstitution with a cut size (for divisor collection) of 8 and a maximum dependency circuit size of 20. Columns "A" show the number of free AIG nodes (nodes not in any boxes) and columns "A+B" show the number of free AIG nodes plus the number of boxes, which resembles the size of the mapped network using a standard cell library consisting of only AND2, half adder, and full adder. Columns "Time" show the optimization time in seconds.

It is observed that optimization with transparent-boxing achieves the most reduction in both the number of free ANDs and the mapped network size. The runtime for black-boxing is the fastest, because many nodes are not seeable by the optimization algorithm, thus reducing computational effort for simulating, verifying, and substituting. However, for the same reason, the optimization quality is the worst.

B. Full-Flow Synthesis Results

The motivation to preserve special logic blocks is that their existence improves physical metrics that are not measurable during technology-independent optimization. Results after a full-flow synthesis are necessary to demonstrate this effect but are not available with open-source academic tools. Thus, in the second experiment, we integrate the discussed boxing techniques, as well as the proposed wire-based resubstitution for transparent-boxed networks, into an industrial flow. Another advantage of experimenting with an industrial tool is that, by starting from RTL netlists, we have access to the practical sources of logic blocks to be boxed, such as complex flops and MUXes.

In Table II, averaged and normalized results on 13 industrial benchmarks synthesized with an industrial flow are presented. The baseline is a vanilla flow without attempting to preserve special logic blocks. Physical metrics on timing (total negative slack, TNS, and worst negative slack, WNS), area, power, and routing (wire length, WL) are compared, where the average delta (Avg.) and the numbers of wins and losses (W/L) among the 13 benchmarks are listed. All of these metrics are better with a smaller value (i.e., a negative delta is good). In the column "Transparent-boxing", the wire-based resubstitution algorithm proposed in Section IV-A is additionally applied after black-boxed optimizations, with black boxes being transformed into transparent boxes.

We observe that most metrics improve over the baseline with boxing techniques, proving the claim that preserving specific logic components is indeed beneficial for physical metrics. Although WNS degrades on average, it is actually improved in more benchmarks than where it degrades. Comparing transparent-boxing against black-boxing, the most prominent improvement lies in the timing metrics. This is reasonable, as the wire-based resubstitution tries to rewire a box input pin with a transitive fanin node, which may have a better arrival time than the original node. The area metric improves in most benchmarks, even though only by about 0.5% on average. This is likely because, as discussed before, the wire-based resubstitution does not ensure a reduction on AIG size. It simply accepts any possible substitution, and zero-gain moves are allowed. Even though the metrics on power and wire length do not improve as much with transparent-boxing as with black-boxing, they still improve by 1.27% and 2.36%, respectively, over the baseline, and the improvements are consistent in most benchmarks. In conclusion, migrating from black-

boxing to transparent-boxing allows more aggressive optimizations being done to further improve on timing and area, while compromising marginally on the other physical metrics as a drawback.

As the industrial synthesis flow is complicated and includes various heuristics, the results could be affected by different factors. The presented experiment does not involve tuning on hyperparameters or improvements to the heuristics. We simply added the boxing strategies to an existing flow. Thus, it is very promising to see improvements in most metrics on most benchmarks, and without outliers.

VI. Conclusions

In this paper, we discuss methods to "box" special logic blocks in technology-independent optimization to preserve them. We argue that don't-care-based optimization algorithms would take advantage of the information of the boxed logic, thus we propose the transparent-boxing technique to enhance optimization quality with boxes. Especially for such, we propose a novel wire-based resubstitution algorithm tailored for finding hidden optimizations in transparent-boxed networks that are otherwise hard to find by other existing optimizations. We also discuss potential problems and details to pay attention to in the implementation of the data structure of black and transparent boxes. Finally, our experimental evaluations show that preserving specific logic blocks—either by black-boxing or transparent-boxing—indeed improves on the physical metrics in an industrial synthesis flow. Moreover, compared to black-boxing, transparent-boxing achieves better optimization quality thanks to the additional information retained by the transparent boxes.

Simple technology-independent representations like AIGs have been shown to be an efficient, scalable and powerful data structure for logic optimization in academic tools. [14] However, in the industrial context, with consideration of physical metrics and information on complex logic components, our experimental evaluation shows that vanilla AIGs are perhaps too naive. Boxing techniques play an essential role in many industrial synthesis tools. The main reason for this is the well-known gap between technology-independent representations and the physical implementations. Various approaches have been explored to close this gap, such as performing early mapping rounds in generic synthesis to provide more accurate estimations on the real costs. These approaches are often complicated and time-consuming. Thus, it is encouraging to see that such a simple technique as transparent-boxing can already help in this context.

References

[1] A. Mishchenko and R. Brayton, "Scalable logic synthesis using a simple circuit structure," in *Proc. IWLS*, vol. 6, 2006, pp. 15–22.

[2] K. Keutzer, "DAGON: technology binding and local optimization by DAG matching," in *Proceedings of the 24th ACM/IEEE Design Automation Conference. Miami Beach, FL, USA, June 28 - July 1, 1987*, 1987, pp. 341–347.

[3] A. T. Calvino and G. D. Micheli, "Technology mapping using multi-output library cells," in *IEEE/ACM International Conference on Computer Aided Design, ICCAD 2023, San Francisco, CA, USA, October 28 - Nov. 2, 2023*. IEEE, 2023, pp. 1–9.

[4] B. L. C. Barzen, A. Reais-Parsi, E. Hung, M. Kang, A. Mishchenko, J. W. Greene, and J. Wawrzynek, "Narrowing the synthesis gap: Academic FPGA synthesis is catching up with the industry," in *Design, Automation & Test in Europe Conference & Exhibition, DATE 2023, Antwerp, Belgium, April 17-19, 2023*. IEEE, 2023, pp. 1–6.

[5] Q. Wu, M. Pedram, and X. Wu, "Clock-gating and its application to low power design of sequential circuits," *IEEE Transactions on Circuits and Systems I: Fundamental Theory and Applications*, vol. 47, no. 3, pp. 415–420, 2000.

[6] T. Kutzschebauch, "Efficient logic optimization using regularity extraction," in *Proceedings of the IEEE International Conference On Computer Design: VLSI In Computers & Processors, ICCD '00, Austin, Texas, USA, September 17-20, 2000*. IEEE Computer Society, 2000, pp. 487–493.

[7] S.-Y. Lee, H. Riener, A. Mishchenko, R. K. Brayton, and G. De Micheli, "A simulation-guided paradigm for logic synthesis and verification," *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.*, vol. 41, no. 8, pp. 2573–2586, 2022.

[8] A. Tempia Calvino and G. De Micheli, "Scalable logic rewriting using don't cares," in *Proceedings of DATE*, 2024.

[9] H. Riener, E. Testa, W. Haaswijk, A. Mishchenko, L. G. Amarù, G. D. Micheli, and M. Soeken, "Scalable generic logic synthesis: One approach to rule them all," in *Proceedings of the 56th Annual Design Automation Conference 2019, DAC 2019, Las Vegas, NV, USA, June 02-06, 2019*. ACM, 2019, p. 70.

[10] M. Soeken, H. Riener, W. Haaswijk, E. Testa, B. Schmitt, G. Meuli, F. Mozafari, S.-Y. Lee, A. Tempia Calvino, D. S. Marakkalage, and G. De Micheli, "The EPFL logic synthesis libraries," 2022. [Online]. Available: http://arxiv.org/abs/1805.05121

[11] L. Amarú, P.-E. Gaillardon, and G. De Micheli, "The EPFL combinational benchmark suite," in *Proceedings of IWLS*, 2015.

[12] R. K. Brayton and A. Mishchenko, "ABC: An academic industrial-strength verification tool," in *Computer Aided Verification, 22nd International Conference, CAV 2010*, 2010, pp. 24–40.

[13] A. Mishchenko, B. Sterin, and R. Brayton, "Structural reverse engineering of arithmetic circuits."

[14] A. Mishchenko and R. Brayton, "Scalable logic synthesis using a simple circuit structure," in *Proc. IWLS*, vol. 6, 2006, pp. 15–22.